SEASUG Paper 42-2026

A Domain-Specific Computer Language for Market Monitoring: From Domain Logic to SAS Code

Eric Grasby[1,2]; Daniel Berleant[1]; Maria CB Gingras[3]

[1]UA Little Rock; [2]Southwest Power Pool (SPP); [3]Consultant

ABSTRACT

This paper presents a proof-of-concept design of a domain-specific language (DSL) for use in wholesale electric market monitoring and applies the DSL to an industry use case. Wholesale electric markets competitively dispatch power plants, while respecting the constraints of the electric grid, to minimize the cost of electricity produced. Market monitors are charged with analyzing data from these markets to screen for prohibited behavior among market participants to protect consumers from unduly high prices and to maintain the reliability of the electric grid.

To fulfill their duties, market monitors must extract, transform, and analyze data from multiple sources, usually without sophisticated programming skills. Additionally, electric market data is complex and often fragmented, leading to issues that transcend simple programming problems. Together, these issues create a barrier for market monitors to efficiently and accurately complete their duties. This paper presents one viable solution to both problems: a DSL that allows a market monitor to declare the metrics that they wish to analyze and a transpiler that translates their input into valid code. This DSL leverages SAS® data sets for data management and SAS procedures, DATA step programming, and SAS macros to perform transformations.

Finally, this paper illustrates this proposed design by demonstrating how a user can use the DSL and generate SAS code to fulfill common and critical market monitoring tasks.

Keywords*: domain-specific language, analyst abstraction, energy market, market monitoring, transpiler*

INTRODUCTION

Electricity is a ubiquitous commodity; its generation, transmission, and use is present in almost every aspect of modern society. Commonly recognized as the “world’s largest machine” [1], the North American Bulk Electric System (BES) is one of the most important systems to the lives of virtually every resident of the United States, Canada, and a portion of Mexico. Organized electric markets, which use the BES to transmit electricity, are constructs that allow a region to pool electric generating resources across many market participants and centrally commit and dispatch these resources to reliably and economically deliver energy to consumers.

Like any market, electric markets are subject to bad actors. However, when market participants engage in manipulative or fraudulent behavior in the electric market, such behaviors can not only impact prices but threaten the physical security of the electric grid and disrupt electric supply to end consumers. One such example is the fraud perpetrated by the *Enron Corporation* in the early 2000s. In the aftermath of the Enron scandal [2], the Federal Energy Regulatory Commission (FERC) mandated that all organized wholesale electric markets in the United States be independently monitored by an energy market monitoring authority[1]. This currently includes the California Independent System Operator (CAISO), Southwest Power Pool (SPP), Midcontinent Independent System Operator (MISO), Pennsylvania, New Jersey, Maryland (PJM), New York Independent System Operator (NYISO), and the Independent System Operator of New England.

Market monitors are the individuals tasked with fulfilling the market monitoring mandate outlined in *CFR 18 35.41*[2]. This includes reporting on the state of the wholesale electric market, investigating instances of fraud and manipulation, and reviewing and advising on existing and proposed market rules to ensure these markets remain fair, efficient, and open access. Fulfilling this mandate requires not only expertise in electric and market policy concepts, but also demands the ability to extract, transform, and analyze data. This requires technical skills in programming and data analysis.

The combination of knowledge and skills combined with the complex nature of wholesale electric markets leads to several information quality problems, including analysts attempting to manage “information overload” [3] (which impacts the accuracy of reported figures), siloed data stores, and undocumented schemas. Such quality problems

[1] The Electric Reliability Council of Texas (ERCOT), while monitored by a wholesale electric market monitoring firm, does not report to FERC and is excluded from this list.

[2] CFR 18 35.41 mandates four rules concerning electricity market behavior, including: 1) generating unit operation, 2) factual communication with FERC, 3) factual price reporting, and 4) a five-year records retention period [4].

erode trust in official reports and can place undue burden on analysts.

The design outlined in this paper intends to lower the programming burden of market monitoring and empower market monitors to spend more time on domain-specific analysis and less time on developing and debugging code. To these ends, we present a foundation for a domain-specific language (DSL) that allows market monitors to express their analytics in notations better suited to energy data. These notations are then automatically translated into valid SAS code which can process the energy market data on behalf of the analyst.

BACKGROUND

The business problem: why a domain-specific language?

The BES itself is one of the world's largest, most operationally complex machines. In the organized electricity markets, a Regional Transmission Organization (RTO) or Independent System Operator (ISO) not only manages the physical grid, ensuring the reliable utilization of the thousands of resources, transformers, transmission lines, and other grid components, but also performs long term transmission planning, studies the addition and retirement of generating resources, commits and dispatches generating resources to minimize the cost of electricity, and settles all wholesale market transactions. A natural result of the diverse roles the RTOs and ISOs play across different time horizons is a complex data infrastructure that is often fragmented. To effectively and holistically analyze the wholesale electric market, market monitors must routinely interact and combine data from these data sets, often across different databases and without clear primary keys and documented schemas.

Market monitoring authorities can and do leverage many analytical software packages to help them with wrangling and analyzing market data. Many of these analytical ecosystems (such as SAS) provide an abstraction layer specifically designed to allow a user to utilize simplified programming syntax with which to process, transform, and analyze data. This substantially lowers the programming skills required to perform market monitoring duties. However, the nature of electric markets and the data which market monitors must utilize create their own layer of complexity that transcends simple programming problems.

Many of these issues are *semantic problems*, requiring an individual to encode their domain expertise into scripts (i.e. SQL or SAS). Specialized analytical methods for electricity market analysis continue to be an active area of research [5]. A simple, yet prevalent, example in energy markets involves how timestamps are interpreted across time zones. Because energy market transactions can span across time zones, timestamps on these transactions may be recorded in local time, adjacent market time, or Coordinated Universal Time (UTC/GMT). If the time zone metadata is not readily apparent, analytical queries may produce *fundamentally incorrect results*, despite being written as *syntactically valid*.

Modern analytics platforms provide mechanisms for performing these conversions, but they tend to rely on the user understanding and correctly applying the domain-level semantics associated with the data (using the appropriate time zone, for example). This is particularly problematic when knowledge of such nuances in the data are not formally documented (as often they are not) and tend to leave the institution when the knowledge holder leaves. This also represents risk to a market monitoring authority's ability to serve its mandate.

A DSL can leverage metadata and domain-aware rules, transparently handling these operations without requiring the user to manually account for such details. The proposed DSL is not intended to replace existing analytics platforms, like SAS, but instead provides a semantic layer that reduces knowledge fragmentation and mitigates information quality risks.

Why SAS®?

The primary justification for developing a DSL that works with SAS as its target coding language is its current prevalence of use among market monitors. However, using SAS as the target language for this DSL comes with further advantages. While the design and implementation of this language provide a parser and language transpiler to effectuate the execution of the language, there are other components that are required to provide users with a functional analysis system. These main components include a data processing engine, a reliable data typing scheme, and capabilities for data visualization and basic cartography.

The SAS ecosystem satisfies each of these system requirements. Base SAS and SAS/ACCESS provide mechanisms for processing data, as well as importing and exporting that data from a variety of external systems (including relational databases). When data is loaded from a source system, SAS houses that data in SAS datasets. These datasets, in turn, provide a reliable and universal API for data processing across all other SAS components. Base SAS also ships with a data typing system, so the upstream DSL does not have to enforce strict typing as part of the syntax (which translates to less cognitive burden on the user).

Additionally, the Output Delivery System (ODS) that ships with SAS provides tools for generating and formatting both

data visualization graphs and spatial maps. The ODS principally provides this functionality through the SGPLOT and SGMAP procedures, respectively.

With each of these infrastructural components available within one platform, a DSL can remain focused on allowing the user to write intuitive data logic. It then offloads any technical complexity to SAS and returns results and error messages back to the user.

DELPHI STUDY

To identify relevant use cases during the implementation of this DSL, the researchers chose to proctor a Delphi study[3]. Participation in the Delphi study was voluntary but limited to members[4] of the Energy Intermarket Surveillance Group (EISG)[5]. This requirement allowed us to establish a definition of "expert opinion" provided by each participant. Plainly, this ensured that the survey questions posed in this study were answered by the same individuals that may use a purpose-made DSL for energy market analytics. The study was conducted in two (2) rounds, where a round corresponds to a group of survey questions that were provided to the participants. These included a list of free-form text responses, multiple choice questions, and questions provided in a Likert format.

Panel Composition

At the time of administering this study, the EISG held a roster of 131 active members. Round #1 yielded a roughly 15% response rate (19 confirmed individuals participated in the first round). Only 11 of the original 19 participants elected to respond to round #2 (roughly a 58% retention rate of the original participants). Of the panelists, 16 of the EISG members were US-based market monitors. The remaining 3 worked for an internationally based market monitoring unit. Table *1* displays the professional role mix of the panelists, indicating that market monitoring analysts make up the largest portion of the panelists, with a mix of other roles including management, economists, data scientists, and a developer.

Role	Count of Role in Panel	Percentage (%)
Market Monitor (Analyst)	13	68%
Management	4	21%
Economist	3	16%
Data Scientist	2	11%
Developer/IT	1	5%

Table 1. Professional Role Mix of Delphi panel[6]

In addition to a representational mix of professional market monitors, the Delphi participants also varied in their "length of tenure" at a market monitoring position. Table *2* identifies the tenure ranges of the participants. Panelists ranged in experience from fresh analysts (0-2 years, 16% of the initial panel) to a full career in energy market monitoring (greater than 20 years of experience, 11% of the initial panel). This panelist cross-section helps to ensure that the requirements developed for the DSL are applicable to a wide range of users.

Tenure Range	Participant Count	Percentage (%)
0-2 years	3	16%
5-10 years	6	32%
10-15 years	7	37%
15-20 years	1	5%
Greater than 20 years	2	11%

Table 2. Tenure range of Delphi Panel

[3] A Delphi study "is a research survey technique used as a way of collecting data from respondents within their domain of expertise." [6] Put plainly, it is a panel of experts that anonymously respond to survey questions. Once the results are processed by the researcher, they are provided back to the panelists (in anonymized format to minimize bias) so that the panelists can achieve a consensus.

[4] The research protocol for this Delphi study was approved by the UA Little Rock Institutional Review Board. The research protocol and copy of the waiver of informed consent can be found in this GitHub repository: https://github.com/echarlesgrasby/dissertation/tree/master/ext.

[5] The Energy Intermarket Surveillance Group (EISG) is the international, professional, conference of energy market monitors. EISG membership is determined by virtue of working for an MMU that is a member of the EISG.

[6] Participants were allowed to select more than one role that applied to them (in Table 1); multiple roles may be represented by the same participant.

Thematic Coding

Of the questions posed during the Delphi study, the following are most applicable to determining the features of the language:

- Q5: "What are the primary data challenges that you face in market monitoring?"
- Q7: "Explain the data sources that you read from and write to in the course of your normal work."
- Q41: "If you had a set of commands at your disposal that could perform often tedious tasks that you have to do in your role, what commands would you want?"

The participant responses were processed using *thematic coding* to identify the common data needs across the market monitoring discipline. The researchers analyzed individual responses for each question (49 total responses for the three questions discussed in this paper) and assigned initial codes[7] to each response. The initial codes were then grouped into overarching themes, from which the language requirements are derived. Table 3, Table 4, and Table 5 show the final themes identified from each question, including the mention frequency from the participants, for Delphi study questions 5, 7, and 41, respectively.

Theme	Number of Mentions
Data Quality and Literacy	6
Data Integration	4
Data Volume and Scalability	4
Metadata, Discovery, and Documentation	4
Infrastructure and Computing Resources	2
Analytics Lifecycle Management	1
Visualization Creation and Reporting	1
Data Governance	1

Table 3. Coded Themes from Question 5

Theme	Number of Mentions
Database	15
CSV	9
Excel	4
Vendor Data	4
Public Data	4
API	4
SAS	2
XML	1

Table 4. Coded Themes from Question 7

Theme	Number of Mentions
Market Surveillance and Monitoring	7
Data Exploration and Querying	4
Validation and Quality Assurance	2
Portfolio and Resource Analysis	2
Metadata and Domain Knowledge	1
Reporting and Automation	1

Table 5. Coded Themes from Question 41

Derived requirements and use cases

Once thematic coding was completed, the researchers used the resultant themes to derive basic requirements and possible use cases for the language proof-of-concept and implementation. These requirements are intentionally broad, as much of the functionality of this system is also determined by what a specific user base (group of market monitors) would request. Table 6 presents the derived requirements, along with the passing criteria, that is, the criteria that the system should meet to ultimately satisfy the requirements.

[7] *Thematic codes* are short descriptors attached to qualitative data that are used to identify the meaning of a phrase or response.

ID	Requirement	Passing Criteria
1	The language shall be declarative. A user can define what they want the analysis and/or output to be, and the transpiler generates code in the target language to accomplish the task.	The user can write a script in the language's declarative grammar and receives transpiled code in the target language.
2	The language, where possible, should infer metadata needed for data selection, transformations, and filter criteria from a metadata repository.	The user can use the language to define metadata (e.g. tags) and associate it with a data set.
3	The language shall, at a minimum, support database queries and CSV files as both read and write targets *(the leading 2 data sources mentioned by Delphi participants).*	A user can specify a database connection, CSV file paths, or both, and use these as sources and destinations for sourcing and sinking data.
4	The language shall allow the user to generate textual reports of pre-defined metrics (a*s an example, analyzing resources by different dimensions, fuel-type, time period, capacity, etc.).*	The user can calculate a pre-defined metric for an operating period and view the report.
5	The language shall allow the user to generate simple visualizations of pre-defined metrics to support market reporting *(possible examples include generating fleet fuel-mix or capacity of generation on outage or locating generating facilities or constraints on a cartographic map).*	The user can create certain pre-defined visualizations that provide metrics for market reporting.

Table 6. Derived DSL Requirements

From these requirements, the following use cases were selected for this paper. These are intended to show that the proof of concept can support the listed requirements.

- Data loading
- Beginner-friendly data traceability
- Basic data visualization generation

DOMAIN-SPECIFIC LANGUAGE DESIGN

The sample language that we propose, *MLang* ("Monitor Language"), uses a simple, declarative syntax that users of SQL may find familiar. See Appendix A for an example of the Lark grammar used to build out the initial functionality of MLang, and Appendix B for a sample MLang program. The language tooling uses basic logic to determine if SAS code should be written "on the fly" (for simple statements) or import pre-written macro templates for complex or multi-step operations.

```
title "Results of Tag Search: %sysfunc(datetime(), datetime20.)";
PROC SQL;

    SELECT * FROM
    (
        select v1.libname as "Library"n, v1.memname as "Data Set"n, v1.name as "Column Name"n, v1.xattr as "Tag Name"n, v1.xvalue as "Tag Value"n
        from sashelp.vxattr v1  /* search for data set tags */
        where upper(v1.xattr) = '{{TAG_NAME}}'
        and v1.name is null
        union all
        select v2.libname as "Library"n, v2.memname as "Data Set"n, v2.name as "Column Name"n, v2.xattr as "Tag Name"n, v2.xvalue as "Tag Value"n
        from sashelp.vxattr v2  /* search for field-level tags*/
        where upper(v2.xattr) = '{{TAG_NAME}}'
        and v2.name is not null
    ) as xx
    ORDER BY "Library"n ASC, "Data Set"n ASC, "Column Name"n ASC, "Tag Name"n ASC;
QUIT;

ods noproctitle;
```

Figure 1. Templated SAS Code ("Tag Search")[8]

[8] This is sourced from `search_tags.sas.tpl`, a template file for querying SAS data sets and/or field names that are tagged with a particular user-defined tag in Mlang (using SAS Extended Attributes).

```
/* import_dataset__basic.sas */

%MACRO IMPORT_DATASET__BASIC(SOURCE=,LIBRARY=WORK,OUT_NAME=,SOURCE_FORMAT=);

    %LET SOURCE=%SYSFUNC(
                    TRANWRD(
                        %SYSFUNC(
                            TRANWRD(&SOURCE., %STR(%"),)
                    ), %STR(%'),)
                );
    %PUT Source file to be loaded is: &SOURCE.;
    %PUT The target library is: &LIBRARY.;

    %LET BASENAME=%SYSFUNC(SCAN(%SYSFUNC(SCAN(&SOURCE., -1, %STR(/))), 1, %STR(.)));
    %LET BASENAMEX=%SYSFUNC(SCAN(&SOURCE., -1, %STR(.)));

    %PUT Basename of target data set is: &BASENAME. and the file extension is: &BASENAMEX.;
    %PUT The incoming data will be loaded to &LIBRARY..&OUT_NAME.;

    PROC IMPORT DATAFILE="&SOURCE."
        OUT=&LIBRARY..&OUT_NAME.
        DBMS=&SOURCE_FORMAT.
        REPLACE;
        GETNAMES=YES;
        GUESSINGROWS=MAX;
    RUN;

%MEND IMPORT_DATASET__BASIC;
```

Figure 2. `import_dataset_basic.sas` (predefined macro)

Figure 1 shows an example of dynamically substituted code (injected directly into the SAS program) while Figure 2 displays `import_dataset_basic.sas`, a pre-written macro (utilizing PROC IMPORT) that MLang leverages to create new data sets from CSV files.

Transpiler Implementation

To implement this domain-specific language, the researchers used two (2) main technologies: Python (including the Lark parser framework [7]) and the SAS® OnDemand for Academics (SODA) [8] environment. The authors created a Python program that defines the grammar of the DSL, builds the lexer[9], parser, and transpiler, and then outputs valid SAS code. The code is then uploaded to SODA to check for execution errors and to retrieve any results. Figure 3 illustrates the process of transpiling MLang code into valid SAS code. In addition to producing a SAS output file, the transpiler program also produces error log and warning log files that help the user determine if there are issues with their MLang logic.

Other researchers have investigated programmatically analyzing SAS syntax before. Khorlo has rigorously demonstrated the viability of transpiling SAS code into Lua (executable via PROC LUA) using ANTLR (Another Tool for Language Recognition – a parser generator built upon the JVM) [9] [10]. Our implementation builds upon this idea from an alternate perspective: transpiling a custom grammar into SAS code.

To support testing the use cases of the transpiler program, the authors utilized data from EIA Form 860 (provided by the Energy Information Authority – EIA). EIA provides a detailed annual report "on the status of existing electric generating plants and associated equipment in the United States…" [11]. Additionally, data provided in the EIA forms contain data quality problems (varying formatting techniques, excessively long field names, inconsistent flags, etc.) that serve to showcase the data quality improvement tools that the DSL provides.

Data provided by EIA Form 860 includes useful parameters such as:

- Utility ownership of generating plants and resources

[9] The terms *lexing* and *parsing* refer to operations in compiler design for working with source code. A lexer takes an input list of tokens and tags those tokens as atomic *lexemes* (e.g. `["Brenda", "was", "the", "best", "programmer", "amongst", "us"]` would be "lexed" as: `["Brenda"(PROPN), "was"(VERB), "the"(DET), "best"(ADJ), "programmer"(NOUN), "amongst"(PREP), "us"(PRON)]`). *Parsing* then takes these lexemes and applies syntactic rules to them to derive a *parse tree* (i.e. identifying "`Brenda`" as the subject of the sentence, "`was`" as the linking verb, etc.).

- Generator physical parameters (nameplate capacity, seasonal capacity, etc.)
- Generator fuel types and technology (e.g. natural gas steam turbine, natural gas combined cycle, etc.)

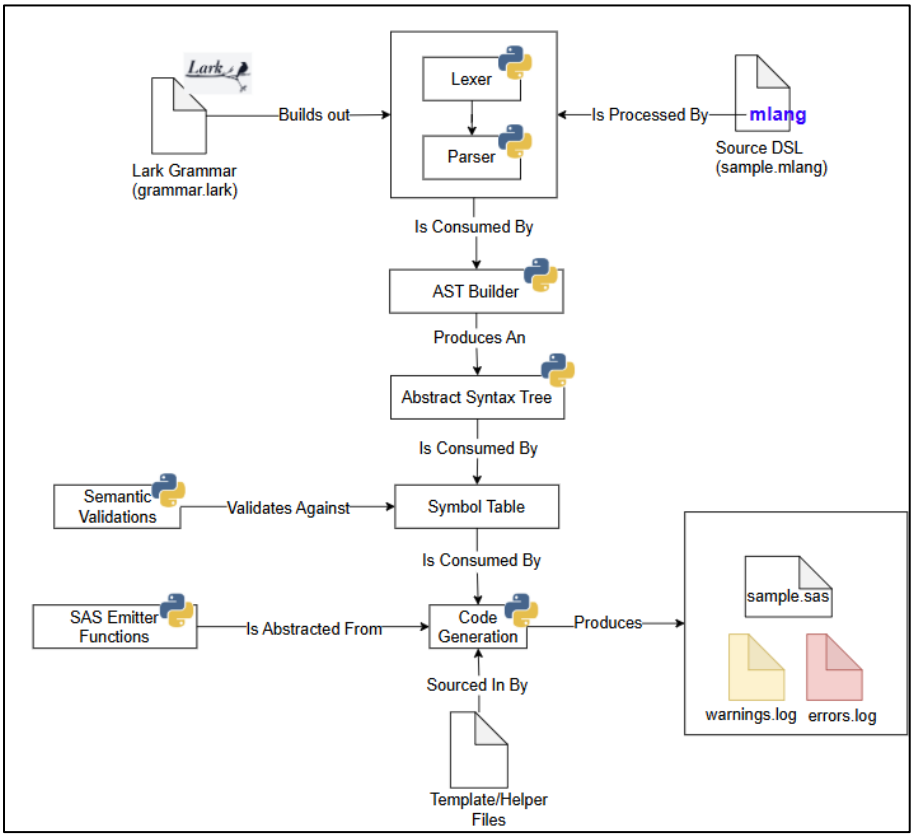


Figure 3. MLang Transpiler Architecture

Execution Examples

Following a run of the transpiler program, an output SAS file is produced. This file can be copied and executed by a user in a SAS environment. The transpiler includes a “preamble” (see Figure 4) with boilerplate code to setup control variables and macros used in the rest of the programs. This functions like the *autocall* feature in SAS, explicitly setting up support code for MLang.

```
1  /* Setup some global folder variables that we can reference throughout the SAS session */
2  %GLOBAL MAIN_FOLDER MACROS_FOLDER FCMP_FOLDER;
3
4  %LET MAIN_FOLDER=%SYSFUNC(PATHNAME(HOME))/sasuser.v94;
5  %LET MACROS_FOLDER=&MAIN_FOLDER./MACROS;
6  %LET FCMP_FOLDER=&MAIN_FOLDER./FCMP;
7
8  %PUT MAIN_FOLDER is: &MAIN_FOLDER.;
9
10 /* Create a MACROS folder if user wishes to include custom macros */
11 data _null_;
12     folder = dcreate("MACROS","&MAIN_FOLDER.");
13     put "MACROS_FOLDER is &MACROS_FOLDER.";
14 run;
15
16 /* Create an FCMP folder if user wishes to create compiled functions */
17 data _null_;
18     folder = dcreate("FCMP","&MAIN_FOLDER.");
19     put "FCMP_FOLDER is &FCMP_FOLDER.";
20 run;
```

Figure 4. Control Variable Setup in Preamble

Figure 5 and Figure 6 show the results of performing a "tag search," querying the SASHELP.VXATTR data set to find objects in SAS that have been labeled with an extended attribute.

```
/* tag search */

title "Results of Tag Search: %sysfunc(datetime(), datetime20.)";
PROC SQL;

    SELECT * FROM
    (
        select v1.libname as "Library"n, v1.memname as "Data Set"n, v1.name as "Column Name"n, v1.xattr as "Tag Name"n, v1.xvalue as "Tag Value"n
        from sashelp.vxattr v1
        where upper(v1.xattr) = 'SOURCE_ORG'
        and v1.name is null
        union all
        select v2.libname as "Library"n, v2.memname as "Data Set"n, v2.name as "Column Name"n, v2.xattr as "Tag Name"n, v2.xvalue as "Tag Value"n
        from sashelp.vxattr v2
        where upper(v2.xattr) = 'SOURCE_ORG'
        and v2.name is not null
    ) as xx
    ORDER BY "Library"n ASC, "Data Set"n ASC, "Column Name"n ASC, "Tag Name"n ASC;
QUIT;

ods noproctitle;
```

Figure 5. Running the Tag Search Code From SODA

Results of Tag Search: 09SEP2026:10:22:18

Library Name	Member Name	Column Name	Extended Attribute Name	Extended Attribute Value
WORK	PLANTS		SOURCE_ORG	Energy Information Authority (EIA)
WORK	UTILITIES		SOURCE_ORG	Energy Information Authority (EIA)

Figure 6. Tag Search Results

Another set of features that are still actively being developed is the generation of visualizations and evaluation of market metrics. Appendix C displays two graphs produced that evaluate the Herfindahl-Hirschman Index (HHI) of SPP's market as a function of generator nameplate capacity. In other words, they depict market concentration by using the production capability of generating resources in SPP as a measure of market share.

In both cases (see utility and plant level capacity visualizations in Appendix C), the HHI (274.7 and 63.3, respectively) is well below 1,000[10], indicating a reasonably unconcentrated market. This HHI analysis was produced using the same SODA environment and test data sets as the MLang proof of concept uses, but the code is still being integrated into the system.

Data Quality Checks

In addition to producing the SAS code, the transpiler also performs multiple data quality checks on the user-provided input files. Figure 7 and Figure 8 display error log snippets from a sample transpiler run. These checks help the user prepare their input data prior to loading into SAS (as a debugging aid). The currently supported quality checks include the following:

- Name checking (for data set and field names)
  - [source] – Do multiple field names collide (or collide once truncated to 32 characters?
  - [collision] – Do multiple data set names collide (or collide once truncated to 32 characters)?
  - [length] – Is the input name restricted to 32 characters or fewer?
  - [spaces] – Does the input name contain spaces?
  - [charset] – Does the input name contain characters that SAS rejects?
  - [reserved-word] – Does the input name shadow a SAS reserved word?
- Source type validation
  - [source-type] – Does MLang support the input source type (e.g. CSV)?

[10] According to the US Department of Justice's definition of HHI [12], any score below 1,000 indicates a reasonably unconcentrated market (i.e. the market is not controlled by a small number of firms).

```
[source] Field names 'FERC_EXEMPT_WHOLESALE_GENERATOR_STATUS' and
'FERC_EXEMPT_WHOLESALE_GENERATOR_DOCKET_NUMBER' in data set 'EIA_SOURCE_PLANTS' collide once truncated
to 32 characters
[source] Field names 'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER' and
'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER_ID' in data set 'EIA_SOURCE_PLANTS' collide once truncated
to 32 characters
[source] Field names 'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER' and
'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER_STATE' in data set 'EIA_SOURCE_PLANTS' collide once
truncated to 32 characters
```

Figure 7. Quality Check Messages in Error Log

```
[source-data-field-name-length] Field name 'PLANNED_NET_WINTER_CAPACITY_DERATE_(MW)' is 39 characters
long; SAS names must be 32 characters or fewer
[source-data-field-name-charset] Field name 'PLANNED_NET_WINTER_CAPACITY_DERATE_(MW)' is not a valid
SAS identifier (must start with a letter or underscore, and contain only letters, digits, and
underscores)
[source-data-field-name-length] Field name 'PLANNED_NEW_NAMEPLATE_CAPACITY_(MW)' is 35 characters
long; SAS names must be 32 characters or fewer
[source-data-field-name-charset] Field name 'PLANNED_NEW_NAMEPLATE_CAPACITY_(MW)' is not a valid SAS
identifier (must start with a letter or underscore, and contain only letters, digits, and underscores)
```

Figure 8. Quality Check Messages in Error Log (cont.)

CONCLUSION

This paper presents a basic foundation for translating a custom, market monitor-focused, domain-specific language into valid SAS code. SAS is a powerful transpiler target for the “Monitor Language”, due to its built-in data management and reporting ecosystem. Currently, MLang supports a limited number of functions, mainly around data loading and error reporting. Since the aim of this paper is to provide proof-of-concept for this DSL, there are several limitations with the current implementation of this solution, namely:

- The execution model of this DSL exists solely as preprocessor directives
- Use of the resultant SAS code requires manual intervention
- The transpiler must be invoked from a command line interface, and interpreting data involves reading log files

Given the identified limitations of this current approach, the authors believe that the following areas are natural opportunities for future work on this system, including (but not limited to):

- Supporting more varied and more complex market monitoring use cases
- Integrating the generated SAS output with a SAS runtime environment
- Adding language server support (LSP) and syntax highlighting to give the user a richer development experience

REFERENCES

[1] Oak Ridge National Laboratory, “Nils Stenvig: Exploring the world’s largest machine |ORNL.” https://www.ornl.gov/blog/nils-stenvig-exploring-worlds-largest-machine.

[2] P. Bondarenko, “Enron Scandal - Downfall and Bankruptcy,” Nov. 2024. Accessed: July 20, 2026. [Online]. Available: https://www.britannica.com/event/Enron-scandal/Downfall-and-bankruptcy

[3] J. Wang, Y. Liu, P. Li, Z. Lin, S. Sindakis, and S. Aggarwal, “Overview of Data Quality: Examining the Dimensions, Antecedents, and Impacts of Data Quality,” *Journal of the Knowledge Economy*, vol. 15, Feb. 2023. Available: https://link.springer.com/article/10.1007/s13132-022-01096-6

[4] Market behavior rules, 18 CFR § 35.41 B (2007). [Online] Available: https://www.ecfr.gov/current/title-18/chapter-I/subchapter-B/part-35/subpart-H/section-35.41

[5] M.-P. Cheong, D. Berleant, and G. Sheblé, “Information Gap Decision Theory as a tool for strategic bidding in competitive electricity markets,” Proceedings, 8th Int. Conf. on Probabilistic Methods Applied to Power Systems, Sept. 12-16, 2004, Ames, Iowa. [Online] Available: https://dberleant.github.io/papers/Mei_PMAPS04a.pdf

[6] S. Chuenjitwongsa, “How to: Conduct a Delphi Study,” *Cardiff University*, 2017. https://www.cardiff.ac.uk/__data/assets/pdf_file/0010/1164961/how_to_conduct_a_delphistudy.pdf

[7] E. Shinan and Contributors, “Lark: A parsing toolkit for Python,” GitHub. [Online]. Available: https://github.com/lark-parser/lark

[8] SAS Institute Inc., *SAS® Studio: OnDemand for Academics.* Cary, NC: SAS Institute Inc. [Online]. Available: https://www.sas.com/en_us/software/on-demand-for-academics.html

[9] I. Khorlo, "Automating Migration from the SAS® Macro Language to the LUA Procedure Using Transpiling," in *Proc. SAS Global Forum 2019*, Dallas, TX, USA, 2019. [Online]. Available: https://support.sas.com/resources/papers/proceedings19/3824-2019.pdf

[10] I. Khorlo, "SASLint: A SAS® Program Checker," in *Proc. SAS Global Forum 2018*, Denver, CO, USA, 2018. [Online]. Available: https://support.sas.com/resources/papers/proceedings18/2543-2018.pdf

[11] U.S. Energy Information Administration, “Form EIA-860 detailed data with previous form data (EIA-860A/860B)”, U.S. Dept. of Energy, Washington, DC. USA, 2024. [Online]. Available: https://www.eia.gov/electricity/data/eia860/.

[12] The United States Department of Justice, “Herfindahl-Hirschman Index,” *Justice.gov*, Jan. 17, 2024. https://www.justice.gov/atr/herfindahl-hirschman-index (accessed Sept. 09, 2026).

ACKNOWLEDGMENTS

We would like to acknowledge SPP and the Energy Information Authority (EIA) that provided sources of publicly available data which made the construction of this project possible. Readers are encouraged to visit the following organizations’ websites for more information on sources of energy data:

- Southwest Power Pool (SPP): https://portal.spp.org/groups/integrated-marketplace
- The Energy Information Authority (EIA): https://www.eia.gov/electricity/

We would also like to acknowledge the members of the EISG that volunteered to participate in the Delphi study. Their responses were instrumental in influencing the design of this proof of concept. Additionally, author EG would like to acknowledge both Mark Rouse and Bartosz “Bart” Jabłoński for inspiring discussions and motivating the use of SAS.

CONTACT INFORMATION

Your questions and comments are encouraged. Contact the authors at:

Eric Grasby; Daniel Berleant
UA Little Rock – School of Computer and Information Science
{ecgrasby, jdberleant} (at) ualr (dot) edu

Maria CB Gingras
maria (at) ochotonaanalytics (dot) com

APPENDIX

See https://github.com/echarlesgrasby/seasug-2026-paper/tree/master/transpiler for any source code referenced in this paper. Other material can be found here.

- Appendix A – Initial Lark Grammar
- Appendix B – Basic Monitor Language (MLang) Program
- Appendix C – Herfindahl-Hirschman Index (HHI) Visualizations

Appendix A: Initial Lark Grammar

```
start: statement+

statement: create_dataset_stmt
         | tag_stmt
         | search_stmt

// CREATE DATASET NAME FROM CSV("/path/to/file.csv");
create_dataset_stmt: "CREATE"i "DATASET"i IDENTIFIER "FROM"i source ";"

source: "CSV"i "(" STRING ")"

// TAG target WITH key = "value";
// TAG target WITH (key1 = "value1", key2 = "value2", );
tag_stmt: "TAG"i target "WITH"i tag_body ";"

target: IDENTIFIER ("." IDENTIFIER)?

tag_body: tag_pair
        | "(" tag_pair ("," tag_pair)* ","? ")"

tag_pair: IDENTIFIER "=" STRING

// SEARCH BY TAG(name) THEN ORDER BY ASC;
search_stmt: "SEARCH"i "BY"i "TAG"i "(" IDENTIFIER ")" order_clause? ";"

order_clause: "THEN"i "ORDER"i "BY"i direction

direction: "ASC"i -> asc
         | "DESC"i -> desc

IDENTIFIER: /[A-Za-z_][A-Za-z0-9_]*/

STRING: /"(?:[^"\\]|\\.)*"/

LANG_COMMENT: /\/\*[\s\S]*?\*\//

// Lark Imports
%import common.WS
%ignore WS
%import common.CPP_COMMENT
%ignore CPP_COMMENT
%ignore LANG_COMMENT
```

Appendix B: Basic Monitor Language (MLang) program

```
/*  ========================================================================
    Author: Eric C. Grasby
    8/5/2026
    ========================================================================
*/

/* Create new dataset: power plants (Source: EIA)*/
CREATE DATASET PLANTS FROM CSV("../data/EIA_860_PLANTS.csv");

/* Tag "EIA_SOURCE_PLANTS" with metadata */
TAG PLANTS WITH SOURCE_ORG = "Energy Information Authority (EIA)";

TAG PLANTS.UTILITY_ID WITH (PRIMARY_KEY = "True" , UNIQUE = "True", );

TAG PLANTS.PLANT_CODE WITH PRIMARY_KEY = "True";

/* Create new dataset: generating utilities (Source: EIA)*/
CREATE DATASET UTILITIES FROM CSV("../data/EIA_860_UTILITIES.csv");

/* Tag "EIA_SOURCE_UTILITIES" with metadata */
TAG UTILITIES WITH SOURCE_ORG = "Energy Information Authority (EIA)";

tag UTILITIES.utility_id with (primary_key = "true", unique = "true"); /*
.mlang is case insensitive! */

/* Create new dataset: generators (Source: EIA) */
CREATE DATASET GENERATORS FROM CSV("../data/EIA_860_GENERATORS.csv");

/* Search for datasets / fields that are tagged with the "SOURCE_ORG" tag */
SEARCH BY TAG(SOURCE_ORG) THEN ORDER BY ASC;

/* line 29 would also have been valid to say: `SEARCH BY TAG(SOURCE_ORG);`
   .mlang would have implicitly ordered results ascending
*/
```

Appendix C: HHI Visualizations

